

Regressão

```
[1]: from IPython.display import Image
```

0.1 Aprendizado de Máquina

É um ramo da Inteligência Artificial que se concentra no desenvolvimento de algoritmos e modelos estatísticos que podem aprender com os dados e fazer previsões.

A **regressão linear** também é um tipo de algoritmo de machine learning, mais especificamente um algoritmo de machine learning supervisionado, que aprende a partir de conjuntos de dados rotulados e mapeia os pontos de dados para as funções lineares mais otimizadas, que podem ser usadas para previsões em novos conjuntos de dados.

Primeiramente, devemos entender o que são algoritmos de machine learning supervisionados. É um tipo de machine learning onde o algoritmo aprende a partir de dados rotulados. Dados rotulados significam que o valor alvo respectivo do conjunto de dados já é conhecido. O aprendizado supervisionado tem dois tipos:

- **Classificação:** Prediz a classe do conjunto de dados com base na variável de entrada independente. A classe são os valores categóricos ou discretos, como a imagem de um animal ser um gato ou um cachorro.
- **Regressão:** Prediz as variáveis de saída contínuas com base na variável de entrada independente, como a previsão de preços de casas com base em diferentes parâmetros, como idade da casa, distância da estrada principal, localização, área, etc.

1 O que é Regressão Linear?

A regressão linear é um tipo de algoritmo de aprendizado de máquina supervisionado que calcula a relação linear entre a variável dependente e um ou mais atributos independentes, ajustando uma equação linear aos dados observados.

Quando há apenas um atributo independente, é conhecida como Regressão Linear Simples, e quando há mais de um atributo, é conhecida como Regressão Linear Múltipla.

Da mesma forma, quando há apenas uma variável dependente, é considerada Regressão Linear Univariada, enquanto quando há mais de uma variável dependente, é conhecida como Regressão Multivariada.

1.1 Por que a Regressão Linear é Importante?

A interpretabilidade da regressão linear é uma força notável.

A equação do modelo fornece coeficientes claros que mostram o impacto de cada variável independente na variável dependente, facilitando uma compreensão mais profunda das dinâmicas subjacentes.

Sua simplicidade é uma virtude, pois a regressão linear é transparente, fácil de implementar e serve como um conceito fundamental para algoritmos mais complexos.

A regressão linear não é apenas uma ferramenta preditiva; ela forma a base para vários modelos avançados. Técnicas como regularização e máquinas de vetores de suporte se inspiram na regressão linear, expandindo sua utilidade. Além disso, a regressão linear é um pilar na testagem de suposições, permitindo que os pesquisadores validem suposições-chave sobre os dados.

1.2 Tipos de Regressão Linear

Existem dois principais tipos de regressão linear:

1. Regressão Linear Simples Este é o tipo mais simples de regressão linear, envolvendo apenas uma variável independente e uma variável dependente. A equação para a regressão linear simples é:

$$Y = \theta_0 + \theta_1 X$$

onde:

- Y é a variável dependente
- X é a variável independente
- θ_0 é o intercepto
- θ_1 é a inclinação

2. Regressão Linear Múltipla

Este tipo envolve mais de uma variável independente e uma variável dependente. A equação para a regressão linear múltipla é:

$$y = \theta_0 + \theta_1 X_1 + \theta_2 X_2 + \dots + \theta_n X_n$$

onde:

- Y é a variável dependente
- $X_1, X_2, \dots, X_n$ são as variáveis independentes
- θ_0 é o intercepto
- $\theta_1, \theta_2, \dots, \theta_n$ são as inclinações

O objetivo do algoritmo é encontrar a melhor equação de linha de ajuste que possa prever os valores com base nas variáveis independentes.

Na regressão, temos um conjunto de registros com valores de X e Y , e esses valores são usados para aprender uma função.

Assim, se você quiser prever Y a partir de um X desconhecido, essa função aprendida pode ser usada. Na regressão, precisamos encontrar o valor de Y , então é necessária uma função que preveja Y contínuo no caso da regressão, dado X como características independentes.

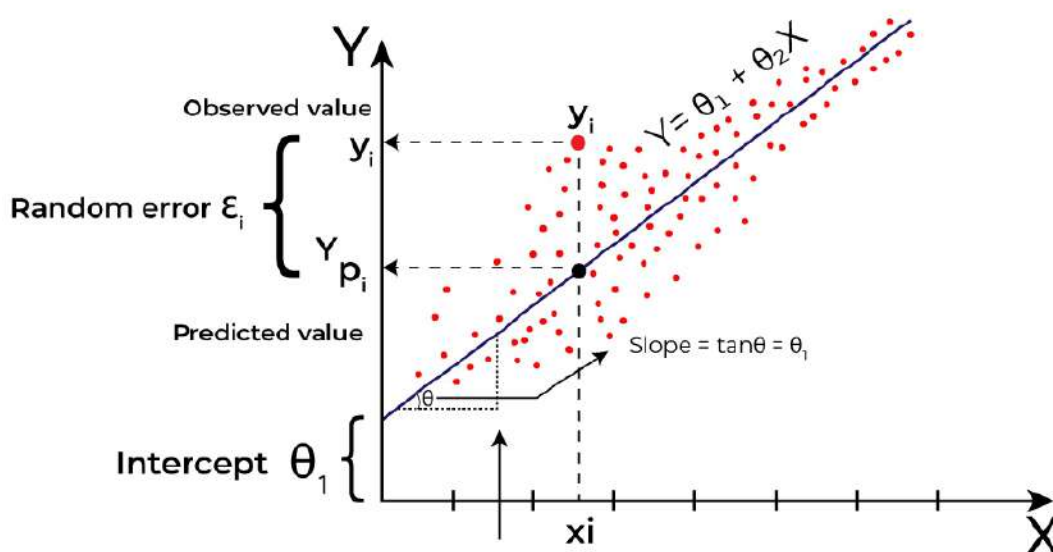
1.3 Qual é a Melhor Linha de Ajuste?

Nosso objetivo principal ao usar a regressão linear é encontrar a melhor linha de ajuste, o que implica que o erro entre os valores previstos e os valores reais deve ser minimizado. A melhor linha de ajuste terá o menor erro.

A equação da melhor linha de ajuste fornece uma linha reta que representa a relação entre as variáveis dependente e independente. A inclinação da linha indica quanto a variável dependente muda para uma unidade de mudança na(s) variável(is) independente(s).

```
[4]: Image(filename='rl.png')
```

[4]:



Na regressão linear, (Y) é chamada de variável dependente ou alvo, e (X) é chamada de variável independente, também conhecida como preditora de (Y).

Existem muitos tipos de funções ou módulos que podem ser usados para regressão. Uma função linear é o tipo mais simples de função. Aqui, (X) pode ser uma única característica ou múltiplas características representando o problema.

A regressão linear realiza a tarefa de prever o valor de uma variável dependente (Y) com base em uma variável independente (X). Portanto, o nome é Regressão Linear. Na figura acima, (X) (entrada) é a experiência de trabalho e (Y) (saída) é o salário de uma pessoa. A linha de regressão é a melhor linha de ajuste para o nosso modelo.

Utilizamos a função de custo para calcular os melhores valores a fim de obter a melhor linha de ajuste, pois diferentes valores para os pesos ou o coeficiente das linhas resultam em diferentes linhas de regressão.

1.4 Função de hipótese em Regressão Linear

Como assumimos anteriormente que nossa característica independente é a experiência, ou seja, (X) e o respectivo salário (Y) é a variável dependente. Vamos supor que haja uma relação linear entre (X) e (Y), então o salário pode ser previsto usando:

$$\hat{Y} = \theta_1 + \theta_2 X$$

OU

$$\hat{y}_i = \theta_1 + \theta_2 x_i$$

Aqui,

- $y_i \in Y (i = 1, 2, \dots, n)$ são rótulos para dados (Aprendizagem supervisionada)
- $x_i \in X (i = 1, 2, \dots, n)$ são os dados de treinamento independentes de entrada (univariados – uma variável de entrada (parâmetro))
- $\hat{y}_i \in \hat{Y} (i = 1, 2, \dots, n)$ são os valores previstos.

O modelo obtém a melhor linha de ajuste de regressão ao encontrar os melhores valores de θ_1 e θ_2 .

- θ_1 : intercept
- θ_2 : coeficiente de x

Uma vez que encontramos os melhores valores de θ_1 e θ_2 , obtemos a linha de melhor ajuste.

Então, quando finalmente estivermos usando nosso modelo para predição, ele irá prever o valor de y para o valor de entrada de x.

1.5 Como atualizar os valores de θ_1 e θ_2 para obter a linha de melhor ajuste?

Para atingir a linha de regressão de melhor ajuste, o modelo visa prever o valor alvo $\hat{Y}$ de modo que a diferença de erro entre o valor previsto $\hat{Y}$ e o valor verdadeiro (Y) seja mínima.

Portanto, é muito importante atualizar os valores de θ_1 e θ_2 para alcançar os melhores valores que minimizem o erro entre o valor (y) previsto (pred) e o valor (y) verdadeiro (y).

$$\text{minimize } \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

1.6 Função custo para Regressão Linear

A função de custo ou função de perda nada mais é do que o erro ou a diferença entre o valor previsto $\hat{Y}$ e o valor verdadeiro (Y).

Na Regressão Linear, a função de custo do Erro Quadrático Médio (MSE) é empregada, que calcula a média dos erros quadrados entre os valores previstos $\hat{y}_i$ e os valores reais y_i .

O objetivo é determinar os valores ótimos para o intercepto θ_1 e o coeficiente da característica de entrada θ_2 , fornecendo a linha de melhor ajuste para os pontos de dados fornecidos.

A equação linear que expressa essa relação é:

$$\hat{y}_i = \theta_1 + \theta_2 x_i$$

A função MSE pode ser calculada como:

$$\text{Função custo } (J) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Utilizando a função MSE, o processo iterativo de descida do gradiente é aplicado para atualizar os valores de θ_1 e θ_2 . Isso garante que o valor do MSE converja para o mínimo global, significando o ajuste mais preciso da linha de regressão linear ao conjunto de dados.

Esse processo envolve ajustar continuamente os parâmetros θ_1 e θ_2 com base nos gradientes calculados a partir do MSE.

O resultado final é uma linha de regressão linear que minimiza as diferenças quadradas totais entre os valores previstos e os valores reais, proporcionando uma representação ótima da relação subjacente nos dados.

1.7 Descida do Gradiente para Regressão Linear

Um modelo de regressão linear pode ser treinado usando o algoritmo de otimização descida do gradiente, modificando iterativamente os parâmetros do modelo para reduzir o erro quadrático médio (MSE) do modelo em um conjunto de dados de treinamento. Para atualizar os valores de θ_1 e θ_2 a fim de reduzir a função de custo (minimizando o valor do RMSE) e obter a linha de melhor ajuste, o modelo usa a Descida do Gradiente. A ideia é começar com valores aleatórios para θ_1 e θ_2 e, em seguida, atualizar iterativamente os valores, alcançando o custo mínimo.

Um gradiente nada mais é do que uma derivada que define os efeitos nas saídas da função com uma pequena variação nas entradas. Vamos diferenciar a função de custo (J) em relação a θ_1 :

$$\begin{aligned} \dot{J}_{\theta_1} &= \frac{\partial J(\theta_1, \theta_2)}{\partial \theta_1} \\ &= \frac{\partial}{\partial \theta_1} \left[\frac{1}{n} \left(\sum_{i=1}^n (\hat{y}_i - y_i)^2 \right) \right] \\ &= \frac{1}{n} \left[\sum_{i=1}^n 2 (\hat{y}_i - y_i) \left(\frac{\partial}{\partial \theta_1} (\hat{y}_i - y_i) \right) \right] \\ &= \frac{1}{n} \left[\sum_{i=1}^n 2 (\hat{y}_i - y_i) \left(\frac{\partial}{\partial \theta_1} (\theta_1 + \theta_2 x_i - y_i) \right) \right] \\ &= \frac{1}{n} \left[\sum_{i=1}^n 2 (\hat{y}_i - y_i) (1 + 0 - 0) \right] \end{aligned}$$

$$\begin{aligned}
&= \frac{1}{n} \left[\sum_{i=1}^n (\hat{y}_i - y_i) (2) \right] \\
&= \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)
\end{aligned}$$

Vamos diferenciar a função de custo J em relação a θ_2 :

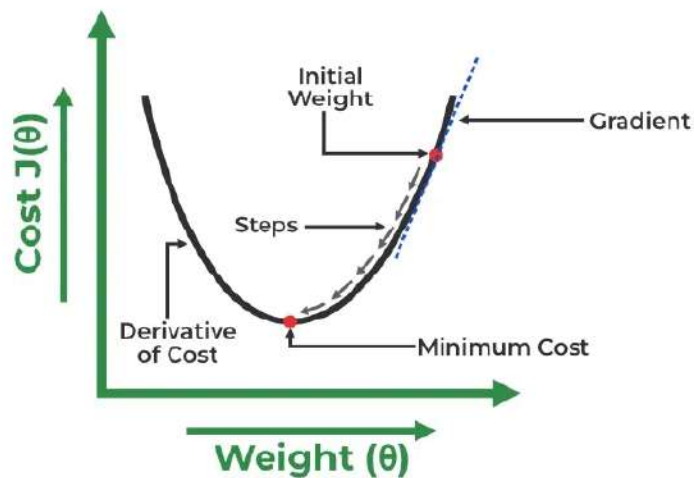
$$\begin{aligned}
J_{\theta_2} &= \frac{\partial J(\theta_1, \theta_2)}{\partial \theta_2} \\
&= \frac{\partial}{\partial \theta_2} \left[\frac{1}{n} \left(\sum_{i=1}^n (\hat{y}_i - y_i)^2 \right) \right] \\
&= \frac{1}{n} \left[\sum_{i=1}^n 2 (\hat{y}_i - y_i) \left(\frac{\partial}{\partial \theta_2} (\hat{y}_i - y_i) \right) \right] \\
&= \frac{1}{n} \left[\sum_{i=1}^n 2 (\hat{y}_i - y_i) \left(\frac{\partial}{\partial \theta_2} (\theta_1 + \theta_2 x_i - y_i) \right) \right] \\
&= \frac{1}{n} \left[\sum_{i=1}^n 2 (\hat{y}_i - y_i) (1 + x_i - 0) \right] \\
&= \frac{1}{n} \left[\sum_{i=1}^n (\hat{y}_i - y_i) (2x_i) \right] \\
&= \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \cdot x_i
\end{aligned}$$

Encontrando os coeficientes de uma equação linear que melhor se ajusta aos dados de treinamento é o objetivo da regressão linear. Ao mover-se na direção do gradiente negativo do Erro Quadrático Médio (MSE) em relação aos coeficientes, os coeficientes podem ser alterados. E o respectivo intercepto e coeficiente de (X) serão ajustados se () for a taxa de aprendizado. Descida do Gradiente

$$1 = 1 - (J_1) = 1 - \left(\frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \right) \quad 2 = 2 - (J_2) = 2 - \left(\frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i \right)$$

[6]: `Image(filename='Gradient-Descent-for-ML-Linear-Regression.png')`

[6]:



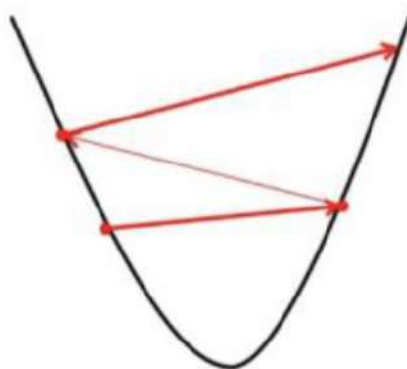
1.7.1 Como escolher a taxa de aprendizagem

A escolha da taxa de aprendizagem correta é muito importante, pois garante que a conversão do Gradiente Descendente em um tempo razoável.

Se escolhermos α como muito grande, o Gradiente Descendente pode ultrapassar o mínimo. Ele pode falhar em convergir ou até mesmo divergir.

```
[3]: Image(filename='big-learning.jpg')
```

```
[3]:
```



Efeito de um grande valor alfa no gradiente descendente

Se escolhermos α como muito pequeno, o Gradiente Descendente dará pequenos passos para atingir os mínimos locais e poderá levar mais tempo para atingir o mínimo.

```
[4]: Image(filename='small-learning.jpg')
```

```
[4]:
```



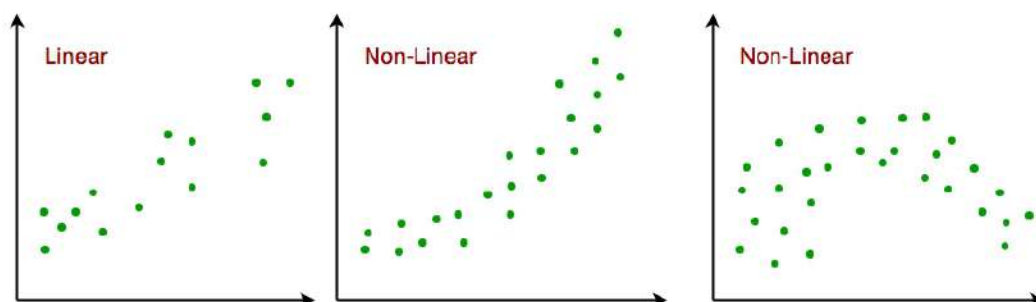
1.8 Pressupostos da Regressão Linear Simples

A regressão linear é uma ferramenta poderosa para entender e prever o comportamento de uma variável. No entanto, ela precisa atender a algumas condições para fornecer soluções precisas e confiáveis.

1. **Linearidade:** As variáveis independente e dependente têm uma relação linear entre si. Isso implica que mudanças na variável dependente seguem aquelas na(s) variável(is) independente(s) de maneira linear. Isso significa que deve haver uma linha reta que possa ser traçada através dos pontos de dados. Se a relação não for linear, então a regressão linear não será um modelo preciso.

```
[7]: Image(filename='python-linear-regression-4.png')
```

```
[7]:
```

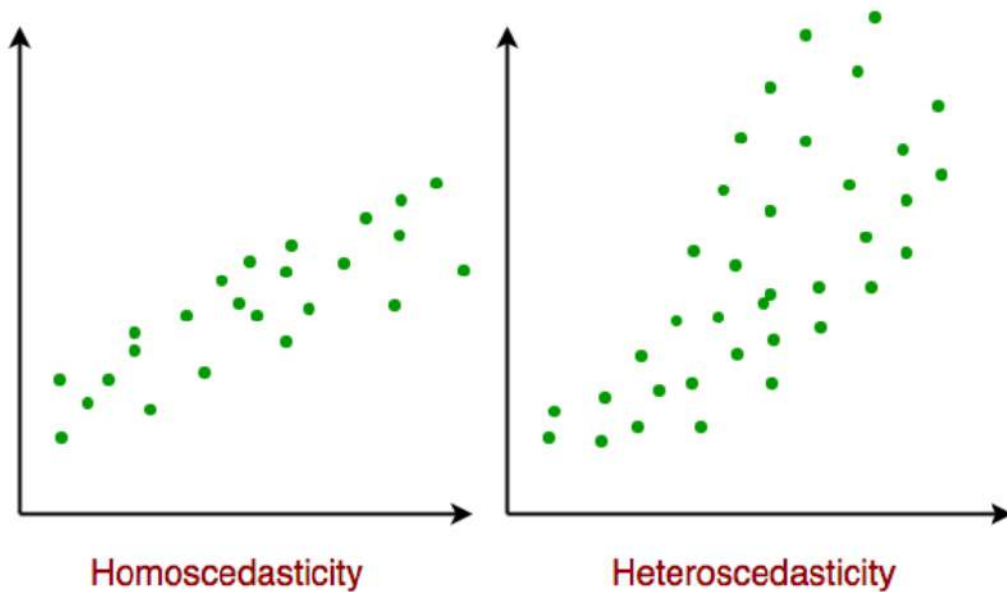


2. **Independência:** As observações no conjunto de dados são independentes umas das outras. Isso significa que o valor da variável dependente para uma observação não depende do valor da variável dependente para outra observação. Se as observações não forem independentes, então a regressão linear não será um modelo preciso.

3. **Homoscedasticidade:** Em todos os níveis da(s) variável(is) independente(s), a variância dos erros é constante. Isso indica que a quantidade da(s) variável(is) independente(s) não tem impacto na variância dos erros. Se a variância dos resíduos não for constante, então a regressão linear não será um modelo preciso.

```
[9]: Image(filename='python-linear-regression-5.png')
```

[9]:



4. **Normalidade:** Os resíduos devem ser distribuídos normalmente. Isso significa que os resíduos devem seguir uma curva em forma de sino. Se os resíduos não forem distribuídos normalmente, então a regressão linear não será um modelo preciso.

1.9 Pressupostos da Regressão Linear Múltipla

Para a Regressão Linear Múltipla, todos os quatro pressupostos da Regressão Linear Simples se aplicam. Além disso, abaixo estão mais alguns:

1. **Ausência de multicolinearidade:** Não há alta correlação entre as variáveis independentes. Isso indica que há pouca ou nenhuma correlação entre as variáveis independentes. A multicolinearidade ocorre quando duas ou mais variáveis independentes são altamente correlacionadas entre si, o que pode dificultar a determinação do efeito individual de cada variável na variável dependente. Se houver multicolinearidade, então a regressão linear múltipla não será um modelo preciso.
2. **Aditividade:** O modelo assume que o efeito das mudanças em uma variável preditora na variável resposta é consistente, independentemente dos valores das outras variáveis. Esse pressuposto implica que não há interação entre as variáveis em seus efeitos na variável dependente.

3. **Seleção de característica:** Na regressão linear múltipla, é essencial selecionar cuidadosamente as variáveis independentes que serão incluídas no modelo. Incluir variáveis irrelevantes ou redundantes pode levar ao overfitting e complicar a interpretação do modelo.
4. **Overfitting:** O overfitting ocorre quando o modelo se ajusta muito de perto aos dados de treinamento, capturando ruídos ou flutuações aleatórias que não representam a verdadeira relação subjacente entre as variáveis. Isso pode levar a um desempenho de generalização ruim em novos dados não vistos.

1.9.1 Multicolinearidade

A multicolinearidade é um fenômeno estatístico que ocorre quando duas ou mais variáveis independentes em um modelo de regressão múltipla são altamente correlacionadas, dificultando a avaliação dos efeitos individuais de cada variável na variável dependente.

Deteção de Multicolinearidade inclui duas técnicas:

- **Matriz de Correlação:** Examinar a matriz de correlação entre as variáveis independentes é uma maneira comum de detectar multicolinearidade. Correlações altas (próximas de 1 ou -1) indicam potencial multicolinearidade.
- **Fator de Inflação da Variância (VIF):** O VIF é uma medida que quantifica o quanto a variância de um coeficiente de regressão estimado aumenta se seus preditores estiverem correlacionados. Um VIF alto (tipicamente acima de 10) sugere multicolinearidade.

1.10 Métricas de Avaliação para Regressão Linear

Uma variedade de medidas de avaliação pode ser usada para determinar a força de qualquer modelo de regressão linear. Essas métricas de avaliação geralmente indicam o quão bem o modelo está produzindo os resultados observados. As medidas mais comuns são:

1.10.1 Erro Quadrático Médio (MSE)

O Erro Quadrático Médio (MSE) é uma métrica de avaliação que calcula a média das diferenças quadradas entre os valores reais e previstos para todos os pontos de dados.

A diferença é quadrada para garantir que diferenças negativas e positivas não se anulem.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Aqui,

- n é o número de pontos de dados.
- y_i é o valor real ou observado para o (i)-ésimo ponto de dados.
- $\hat{y}_i$ é o valor previsto para o (i)-ésimo ponto de dados.

O MSE é uma maneira de quantificar a precisão das previsões de um modelo. O MSE é sensível a outliers, pois grandes erros contribuem significativamente para a pontuação geral.

1.10.2 Erro Médio Absoluto (MAE)

O Erro Médio Absoluto é uma métrica de avaliação usada para calcular a precisão de um modelo de regressão. O MAE mede a diferença absoluta média entre os valores previstos e os valores reais. Matematicamente, o MAE é expresso como:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i|$$

Aqui,

- n é o número de observações
- Y_i representa os valores reais
- $\hat{Y}_i$ representa os valores previstos

Um valor de MAE menor indica melhor desempenho do modelo. Ele não é sensível aos outliers, pois consideramos as diferenças absolutas.

1.10.3 Raiz do Erro Quadrático Médio (RMSE)

A raiz do erro quadrático médio descreve quão bem os pontos de dados observados correspondem aos valores esperados, ou o ajuste absoluto do modelo aos dados.

Em notação matemática, pode ser expresso como:

$$\text{RMSE} = \sqrt{\frac{\text{RSS}}{n}} = \sqrt{\frac{\sum_{i=2}^n (y_i^{\text{real}} - y_i^{\text{predito}})^2}{n}}$$

Em vez de dividir o número total de pontos de dados no modelo pelo número de graus de liberdade, deve-se dividir a soma dos resíduos ao quadrado para obter uma estimativa não tendenciosa. Então, essa figura é referida como o Erro Padrão Residual (RSE). Em notação matemática, pode ser expresso como:

$$\text{RMSE} = \sqrt{\frac{\text{RSS}}{n}} = \sqrt{\frac{\sum_{i=2}^n (y_i^{\text{real}} - y_i^{\text{predito}})^2}{n - 2}}$$

O RMSE não é uma métrica tão boa quanto o R-quadrado. O Erro Quadrático Médio pode flutuar quando as unidades das variáveis variam, pois seu valor depende das unidades das variáveis (não é uma medida normalizada).

1.10.4 Coeficiente de Determinação (R-quadrado)

O R-quadrado é uma estatística que indica quanta variação o modelo desenvolvido pode explicar ou capturar. Ele está sempre na faixa de 0 a 1. Em geral, quanto melhor o modelo se ajusta aos dados, maior é o valor do R-quadrado.

Em notação matemática, pode ser expresso como:

$$R^2 = 1 - \left(\frac{\text{RSS}}{\text{TSS}} \right)$$

- **Soma dos Quadrados dos Resíduos (RSS):** A soma dos quadrados dos resíduos para cada ponto de dados no gráfico ou nos dados é conhecida como soma dos quadrados dos resíduos, ou RSS. É uma medida da diferença entre o resultado observado e o que foi antecipado.

$$\text{RSS} = \sum_{i=2}^n (y_i - b_0 - b_1 x_i)^2$$

- **Soma Total dos Quadrados (TSS):** A soma dos erros dos pontos de dados em relação à média da variável resposta é conhecida como soma total dos quadrados, ou TSS.

$$\text{TSS} = \sum (y - \bar{y}_i)^2$$

A métrica R-quadrado é uma medida da proporção da variância na variável dependente que é explicada pelas variáveis independentes no modelo.

1.10.5 Erro do R-quadrado Ajustado

O R-quadrado ajustado mede a proporção da variância na variável dependente que é explicada pelas variáveis independentes em um modelo de regressão.

O R-quadrado ajustado leva em conta o número de preditores no modelo e penaliza o modelo por incluir preditores irrelevantes que não contribuem significativamente para explicar a variância nas variáveis dependentes.

Matematicamente, o R-quadrado ajustado é expresso como:

$$R^2 \text{ ajustado} = 1 - \left(\frac{(1 - R^2)(n - 1)}{N - K - 1} \right)$$

Aqui,

- n é o número de observações
- k é o número de preditores no modelo
- R^2 é o coeficiente de determinação

O R-quadrado ajustado ajuda a prevenir o overfitting. Ele penaliza o modelo com preditores adicionais que não contribuem significativamente para explicar a variância na variável dependente.

1.11 Implementação de uma Regressão Linear

```
[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.axes as ax
from matplotlib.animation import FuncAnimation
```

```
[3]: data = pd.read_csv('data_for_lr.csv')
data
# removendo valores faltantes
```

```

data = data.dropna()
# conjunto de dados de treinamento
train_input = np.array(data.x[0:500]).reshape(500, 1)
train_output = np.array(data.y[0:500]).reshape(500, 1)
# conjunto de dados de teste
test_input = np.array(data.x[500:700]).reshape(199, 1)
test_output = np.array(data.y[500:700]).reshape(199, 1)

```

```

[4]: class LinearRegression:
    def __init__(self):
        self.parameters = {}

    def forward_propagation(self, train_input):
        m = self.parameters['m']
        c = self.parameters['c']
        predictions = np.multiply(m, train_input) + c
        return predictions

    def cost_function(self, predictions, train_output):
        cost = np.mean((train_output - predictions) ** 2)
        return cost

    def backward_propagation(self, train_input, train_output, predictions):
        derivatives = {}
        df = (train_output - predictions) * -1
        dm = np.mean(np.multiply(train_input, df))
        dc = np.mean(df)
        derivatives['dm'] = dm
        derivatives['dc'] = dc
        return derivatives

    def update_parameters(self, derivatives, learning_rate):
        self.parameters['m'] = self.parameters['m'] - learning_rate * derivatives['dm']
        self.parameters['c'] = self.parameters['c'] - learning_rate * derivatives['dc']

    def train(self, train_input, train_output, learning_rate, iters):
        #initialize random parameters
        self.parameters['m'] = np.random.uniform(0,1) * -1
        self.parameters['c'] = np.random.uniform(0,1) * -1

        #initialize loss
        self.loss = []

        #iterate
        for i in range(iters):

```

```

        #forward propagation
        predictions = self.forward_propagation(train_input)

        #cost function
        cost = self.cost_function(predictions, train_output)

        #append loss and print
        self.loss.append(cost)
        print("Iteration = {}, Loss = {}".format(i+1, cost))

        #back propagation
        derivatives = self.backward_propagation(train_input,
        ↵train_output, predictions)

        #update parameters
        self.update_parameters(derivatives, learning_rate)

    return self.parameters, self.loss

```

```

[5]: #Example usage
linear_reg = LinearRegression()
parameters, loss = linear_reg.train(train_input, train_output, 0.0001, 20)

```

```

Iteration = 1, Loss = 8408.70922055227
Iteration = 2, Loss = 3819.3290491542116
Iteration = 3, Loss = 1737.1221812028
Iteration = 4, Loss = 792.4226180736543
Iteration = 5, Loss = 363.81137909238345
Iteration = 6, Loss = 169.34998441774147
Iteration = 7, Loss = 81.12263345907542
Iteration = 8, Loss = 41.0937858035931
Iteration = 9, Loss = 22.932652483454245
Iteration = 10, Loss = 14.692925378882817
Iteration = 11, Loss = 10.954551263713103
Iteration = 12, Loss = 9.258445985759387
Iteration = 13, Loss = 8.48892034139205
Iteration = 14, Loss = 8.13978490314101
Iteration = 15, Loss = 7.981380967086791
Iteration = 16, Loss = 7.909512124881705
Iteration = 17, Loss = 7.876904350646146
Iteration = 18, Loss = 7.862109366475883
Iteration = 19, Loss = 7.8553960649287715
Iteration = 20, Loss = 7.852349431779728

```

1.12 Aplicações da Regressão Linear

A regressão linear é utilizada em muitos campos diferentes, incluindo finanças, economia e psicologia, para entender e prever o comportamento de uma variável específica. Por exemplo, nas finanças,

a regressão linear pode ser usada para entender a relação entre o preço das ações de uma empresa e seus lucros ou para prever o valor futuro de uma moeda com base em seu desempenho passado.

1.13 Vantagens e Desvantagens da Regressão Linear Vantagens da Regressão Linear

1.13.1 Vantagens

- A regressão linear é um algoritmo relativamente simples, o que a torna fácil de entender e implementar. Os coeficientes do modelo de regressão linear podem ser interpretados como a mudança na variável dependente para uma mudança de uma unidade na variável independente, fornecendo insights sobre as relações entre as variáveis.
- A regressão linear é computacionalmente eficiente e pode lidar com grandes conjuntos de dados de forma eficaz. Ela pode ser treinada rapidamente em grandes conjuntos de dados, tornando-a adequada para aplicações em tempo real.
- A regressão linear é relativamente robusta a outliers em comparação com outros algoritmos de aprendizado de máquina. Outliers podem ter um impacto menor no desempenho geral do modelo.
- A regressão linear muitas vezes serve como um bom modelo de referência para comparação com algoritmos de aprendizado de máquina mais complexos.
- A regressão linear é um algoritmo bem estabelecido com uma rica história e está amplamente disponível em várias bibliotecas de aprendizado de máquina e pacotes de software.

1.13.2 Desvantagens

- A regressão linear assume uma relação linear entre as variáveis dependente e independente. Se a relação não for linear, o modelo pode não ter um bom desempenho.
- A regressão linear é sensível à multicolinearidade, que ocorre quando há uma alta correlação entre as variáveis independentes. A multicolinearidade pode inflar a variância dos coeficientes e levar a previsões instáveis do modelo.
- A regressão linear assume que as características já estão em uma forma adequada para o modelo. Pode ser necessário realizar engenharia de características para transformar as características em um formato que possa ser usado de forma eficaz pelo modelo.
- A regressão linear é suscetível tanto ao overfitting quanto ao underfitting. O overfitting ocorre quando o modelo aprende muito bem os dados de treinamento e falha em generalizar para dados não vistos. O underfitting ocorre quando o modelo é muito simples para capturar as relações subjacentes nos dados.
- A regressão linear oferece poder explicativo limitado para relações complexas entre variáveis. Técnicas de aprendizado de máquina mais avançadas podem ser necessárias para obter insights mais profundos.

A regressão linear é um algoritmo fundamental de aprendizado de máquina que tem sido amplamente utilizado por muitos anos devido à sua simplicidade, interpretabilidade e eficiência. É uma ferramenta valiosa para entender as relações entre variáveis e fazer previsões em uma variedade de aplicações.

No entanto, é importante estar ciente de suas limitações, como a suposição de linearidade e a sensibilidade à multicolinearidade. Quando essas limitações são cuidadosamente consideradas, a regressão linear pode ser uma ferramenta poderosa para análise de dados e previsão.

1.14 Tipos de Regressão Linear

Existem dois tipos principais de regressão linear:

1. **Regressão linear simples:** Envolve a previsão de uma variável dependente com base em uma única variável independente.
2. **Regressão linear múltipla:** Envolve a previsão de uma variável dependente com base em múltiplas variáveis independentes.

1.14.1 Regressão Linear Simples

A regressão linear simples é uma abordagem para prever uma resposta usando uma única característica. É um dos modelos de aprendizado de máquina mais básicos que um entusiasta de aprendizado de máquina conhece. Na regressão linear, assumimos que as duas variáveis, ou seja, variáveis dependente e independente, estão linearmente relacionadas. Portanto, tentamos encontrar uma função linear que preveja o valor da resposta (y) da forma mais precisa possível como uma função da característica ou variável independente (x). Vamos considerar um conjunto de dados onde temos um valor de resposta y para cada característica x:

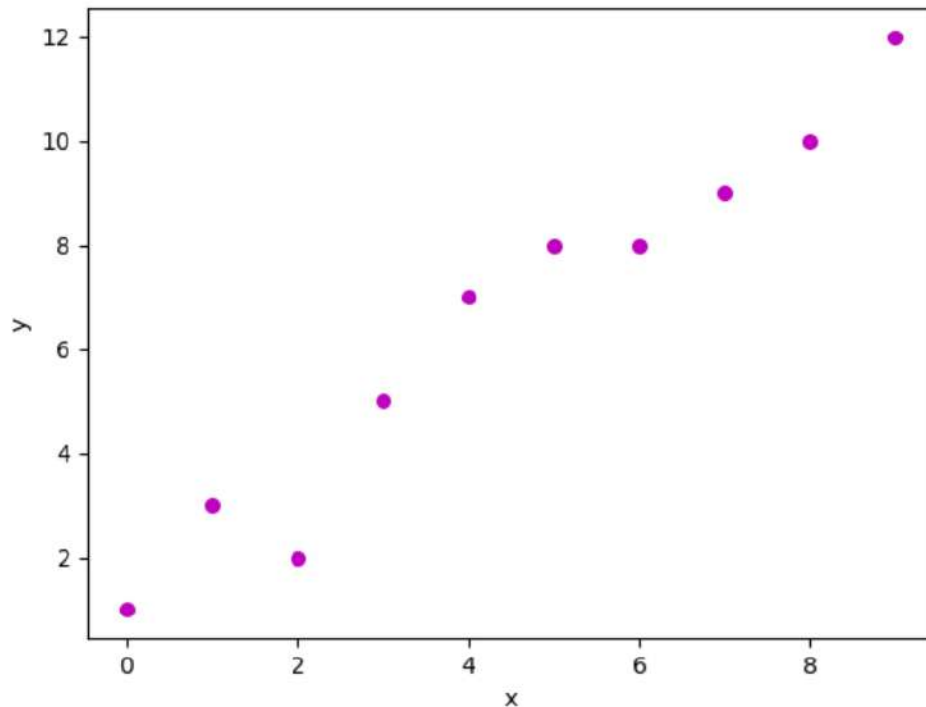
```
[8]: Image(filename='python-linear-regression.png')
```

[8]:

x	0	1	2	3	4	5	6	7	8	9
y	1	3	2	5	7	8	8	9	10	12

```
[9]: Image(filename='python-linear-regression-1.png')
```

[9]:



Para generalidade, definimos:

- x como vetor de características, ou seja, $x = [x_1, x_2, \dots, x_n]$,
- y como vetor de respostas, ou seja, $y = [y_1, y_2, \dots, y_n]$.

para n observações (no exemplo acima, $n=10$). Um gráfico de dispersão do conjunto de dados acima é apresentado abaixo:

Agora, a tarefa é encontrar uma linha que se ajuste melhor ao gráfico de dispersão acima para que possamos prever a resposta para quaisquer novos valores de características (ou seja, um valor de x não presente no conjunto de dados). Esta linha é chamada de linha de regressão. A equação da linha de regressão é representada como:

$$h(x_i) = \beta_0 + \beta_1 x_i$$

Onde:

- $h(x_i)$ representa o valor da resposta previsto para a i -ésima observação.
- β_0 e β_1 são os coeficientes de regressão e representam, respectivamente, a interseção no eixo y e a inclinação da linha de regressão.

Para criar nosso modelo, precisamos estimar os valores dos coeficientes de regressão b_0 e b_1 .

Uma vez estimados esses coeficientes, podemos usar o modelo para prever respostas! Vamos usar o princípio dos Mínimos Quadrados. Considere:

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i = h(x_i) + \epsilon_i \Rightarrow \epsilon_i = y_i - h(x_i)$$

Aqui, ϵ_i é o erro residual na i -ésima observação.

Nosso objetivo é minimizar o erro residual total. Definimos o erro quadrático ou função de custo, (J), como:

$$J(\beta_0, \beta_1) = \frac{1}{2n} \sum_{i=1}^n \epsilon_i^2$$

Nossa tarefa é encontrar os valores de β_0 e β_1 para os quais $J(\beta_0, \beta_1)$ seja mínimo!

Sem entrar nos detalhes matemáticos, apresentamos o resultado aqui:

$$\beta_1 = \frac{SS_{xy}}{SS_{xx}}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

Onde SS_{xy} é a soma das covariâncias de y e x :

$$SS_{xy} = \sum_{i=1}^n (x_i - \hat{x})(y_i - \hat{y}) = \sum_{i=1}^n y_i x_i - n\hat{x}\hat{y}$$

E SS_{xx} é a soma dos desvios quadráticos de x :

$$SS_{xx} = \sum_{i=1}^n (x_i - \hat{x})^2 = \sum_{i=1}^n x_i^2 - n(\hat{x})^2$$

1.14.2 Implementação em Python da Regressão Linear Simples

Podemos usar a linguagem Python para aprender os coeficientes dos modelos de regressão linear. Para plotar os dados de entrada e a linha de melhor ajuste, usaremos a biblioteca matplotlib. É uma das bibliotecas Python mais usadas para plotar gráficos. Aqui está um exemplo de regressão linear simples usando Python:

```
[10]: import numpy as np
import matplotlib.pyplot as plt
```

Função de Estimativa de Coeficientes

Esta função, `estimate_coef()`, recebe os dados de entrada x (variável independente) e y (variável dependente) e estima os coeficientes da linha de regressão linear usando o método dos mínimos quadrados.

1. **Calculando o Número de Observações:** $n = \text{np.size}(x)$ determina o número de pontos de dados.
2. **Calculando as Médias:** $m_x = \text{np.mean}(x)$ e $m_y = \text{np.mean}(y)$ calculam os valores médios de (x) e (y) , respectivamente.
3. **Calculando a Desvio Cruzado e o Desvio em relação a x :** $SS_xy = \text{np.sum}(yx) - nm_ym_x$ e $SS_xx = \text{np.sum}(xx) - nm_xm_x$ calculam a soma dos desvios quadrados entre x e y e a soma dos desvios quadrados de x em relação à sua média, respectivamente.
4. **Calculando os Coeficientes de Regressão:** $b_1 = SS_xy / SS_xx$ e $b_0 = m_y - b_1*m_x$ determinam a inclinação (b_1) e a interceptação (b_0) da linha de regressão usando o método dos mínimos quadrados.
5. **Retornando os Coeficientes:** A função retorna os coeficientes estimados como uma tupla (b_0, b_1) .

```
[11]: def estimate_coef(x, y):
    # número de observações/pontos
    n = np.size(x)
    # média do vetor x e y
    m_x = np.mean(x)
    m_y = np.mean(y)
    # cálculo de desvio cruzado e desvio sobre x
    SS_xy = np.sum(y*x) - n*m_y*m_x
    SS_xx = np.sum(x*x) - n*m_x*m_x
    # cálculo de coeficientes de regressão
    b_1 = SS_xy / SS_xx
    b_0 = m_y - b_1*m_x
    return (b_0, b_1)
```

Função de Plotagem da Linha de Regressão

Esta função, `plot_regression_line()`, recebe os dados de entrada x (variável independente), y (variável dependente) e os coeficientes estimados b para desenhar a linha de regressão e os pontos de dados.

1. **Desenhando o Gráfico de Dispersão:** `plt.scatter(x, y, color = "m", marker = "o", s = 30)` plota os pontos de dados originais como um gráfico de dispersão com marcadores vermelhos.
2. **Calculando o Vetor de Resposta Previsto:** $y_pred = b[0] + b[1]*x$ calcula os valores previstos para y com base nos coeficientes estimados b .
3. **Desenhando a Linha de Regressão:** `plt.plot(x, y_pred, color = "g")` plota a linha de regressão usando os valores previstos e a variável independente x .
4. **Adicionando os Rótulos:** `plt.xlabel('x')` e `plt.ylabel('y')` rotulam o eixo x como 'x' e o eixo y como 'y', respectivamente.

```
[12]: def plot_regression_line(x, y, b):
    # desenhando os pontos atuais em um scatter plot
    plt.scatter(x, y, color = "m", marker = "o", s = 30)
```

```

# predicted response vector
y_pred = b[0] + b[1]*x
# desenhando a regressão linear
plt.plot(x, y_pred, color = "g")
# inserindo os rótulos
plt.xlabel('Tamanho')
plt.ylabel('Custo')

```

```

[13]: def main():
    # Datasets which we create
    x = np.array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
    y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])

    # estimating coefficients
    b = estimate_coef(x, y)
    print("Estimated coefficients:\nb_0 = {} \nb_1 = {}".format(b[0], b[1]))

    # plotting regression line
    plot_regression_line(x, y, b)

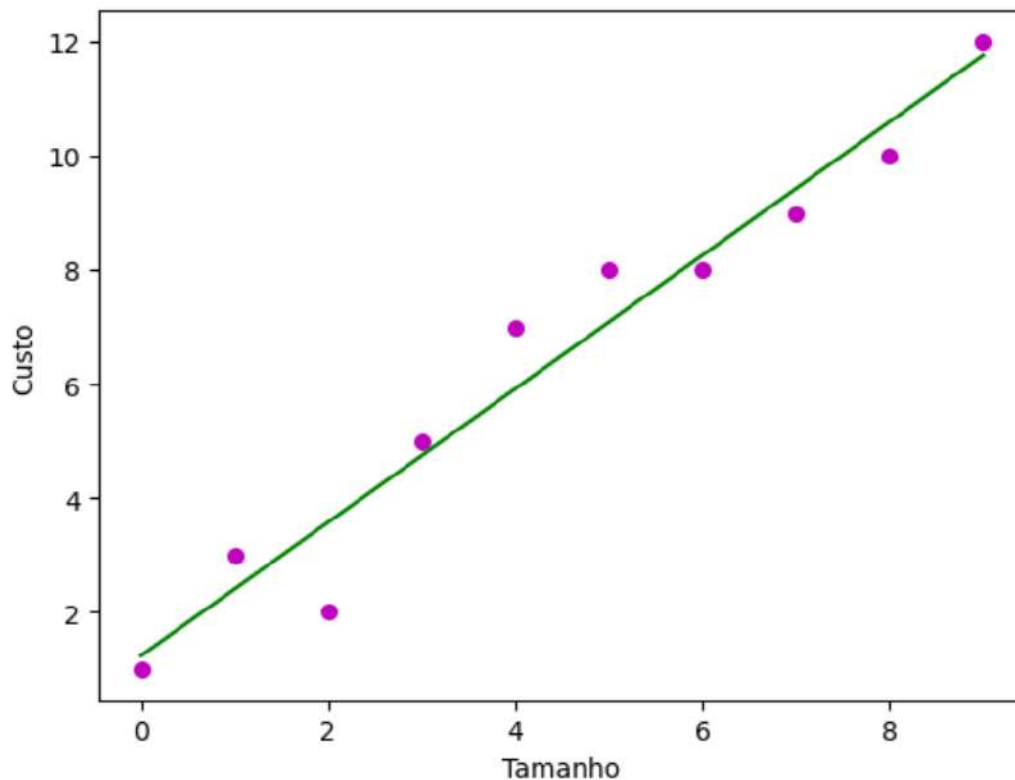
if __name__ == "__main__":
    main()

```

```

Estimated coefficients:
b_0 = 1.2363636363636363
b_1 = 1.1696969696969697

```



1.14.3 Regressão Linear Múltipla

A regressão linear múltipla tenta modelar a relação entre duas ou mais características e uma resposta ajustando uma equação linear aos dados observados.

Claramente, é apenas uma extensão da regressão linear simples.

Considere um conjunto de dados com p características (ou variáveis independentes) e uma resposta (ou variável dependente). Além disso, o conjunto de dados contém n linhas/observações.

Definimos:

- X (matriz de atributos) = uma matriz de tamanho $n \times p$ onde x_{ij} denota os valores da j -ésima característica para a i -ésima observação. Assim,

$$\begin{pmatrix} x_{11} & \cdots & x_{1p} \\ x_{21} & \cdots & x_{2p} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{np} \end{pmatrix}$$

e y (vetor de resposta) = um vetor de tamanho n onde y_i denota o valor da resposta para a i -ésima observação.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

A linha de regressão para p características é representada como:

$$h(x_i) = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}$$

onde $h(x_i)$ é o valor da resposta previsto para a i -ésima observação e $\beta_0, \beta_1, \dots, \beta_p$ são os coeficientes de regressão.

Também podemos escrever:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \epsilon_i$$

ou

$$y_i = h(x_i) + \epsilon_i \Rightarrow \epsilon_i = y_i - h(x_i)$$

onde ϵ_i representa um erro residual na i -ésima observação. Podemos generalizar nosso modelo linear um pouco mais representando a matriz de características X como:

$$\begin{pmatrix} 1 & x_{11} & \dots & x_{1p} \\ 1 & x_{21} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \dots & x_{np} \end{pmatrix}$$

Assim, o modelo linear pode ser expresso em termos de matrizes como:

$$y = X\beta + \epsilon$$

onde,

$$\begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

e

$$\begin{bmatrix} \epsilon_0 \\ \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

Agora, determinamos uma estimativa de β , ou seja, β' usando o método dos Mínimos Quadrados.

Como já explicado, o método dos Mínimos Quadrados tende a determinar β' para o qual o erro residual total é minimizado.

Apresentamos o resultado diretamente aqui:

$$\hat{\theta} = (X'X)^{-1}X'y$$

onde ' representa a transposição da matriz enquanto -1 representa a inversa da matriz.

Conhecendo as estimativas dos mínimos quadrados, β' , o modelo de regressão linear múltipla pode agora ser estimado como:

$$\hat{y} = X\hat{\theta}$$

onde $\hat{y}$ é o vetor de resposta estimado.

1.14.4 Implementação em Python da Regressão Linear Múltipla

Para a regressão linear múltipla usando Python, usaremos o conjunto de dados de preços de casas de Boston.

Importando Bibliotecas

```
[14]: from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets, linear_model, metrics
import pandas as pd
```

Carregando o Conjunto de Dados de Habitação de Boston O código faz o download do conjunto de dados de Habitação de Boston a partir da URL fornecida e o lê em um DataFrame do Pandas (raw_df).

```
[15]: data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
```

Pré-processamento dos Dados Esta etapa extrai as variáveis de entrada (X) e a variável alvo (y) do DataFrame.

As variáveis de entrada são selecionadas de cada linha alternada para corresponder à variável alvo, que está disponível em cada linha alternada.

```
[16]: X = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
y = raw_df.values[1::2, 2]
```

Dividindo os Dados em Conjuntos de Treinamento e Teste Os dados são divididos em conjuntos de treinamento e teste usando a função `train_test_split()` do `scikit-learn`. O parâmetro `test_size` especifica que 40% dos dados devem ser usados para teste.

```
[17]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4,
↳ random_state=1)
```

Criando e Treinando o Modelo de Regressão Linear Isso inicializa um objeto `LinearRegression` (`reg`) e treina o modelo usando os dados de treinamento (`X_train`, `y_train`).

```
[18]: reg = linear_model.LinearRegression()
reg.fit(X_train, y_train)
```

```
[18]: LinearRegression()
```

Avaliando o Desempenho do Modelo Avalia o desempenho do modelo imprimindo os coeficientes de regressão e calculando a pontuação de variância, que mede a proporção da variância explicada. Uma pontuação de 1 indica previsão perfeita.

```
[19]: # coeficientes de regressão
print('Coeficientes: ', reg.coef_)
# pontuação de variância: 1 significa previsão perfeita
print('Pontuação de variância: {}'.format(reg.score(X_test, y_test)))
```

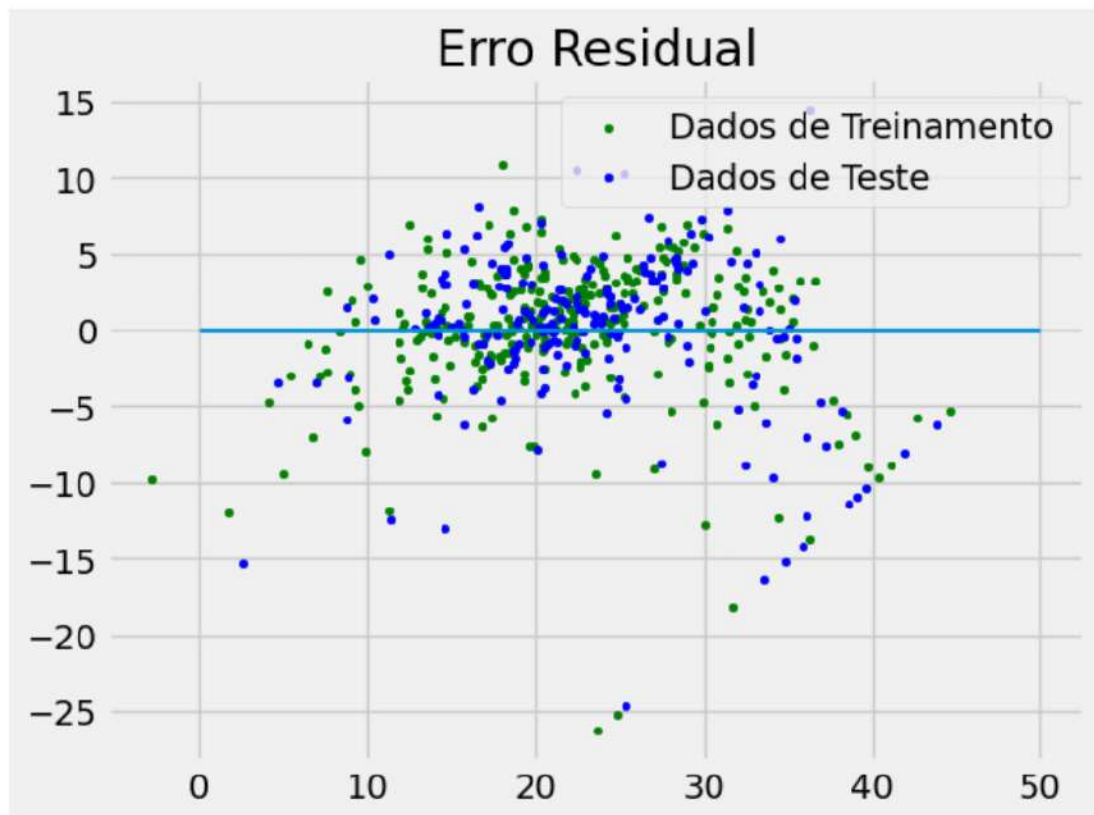
```
Coeficientes: [-8.95714048e-02  6.73132853e-02  5.04649248e-02  2.18579583e+00
-1.72053975e+01  3.63606995e+00  2.05579939e-03 -1.36602886e+00
 2.89576718e-01 -1.22700072e-02 -8.34881849e-01  9.40360790e-03
-5.04008320e-01]
```

```
Pontuação de variância: 0.720905667266174
```

Desenhando Erros Residuais Desenhando e analisando os erros residuais, que representam a diferença entre os valores previstos e os valores reais.

```
[20]: # plotar erro residual
# definir estilo do plot
plt.style.use('fivethirtyeight')
# plotando erros residuais nos dados de treinamento
plt.scatter(reg.predict(X_train), reg.predict(X_train) - y_train,
↳ color="green", s=10, label='Dados de Treinamento')
# plotando erros residuais nos dados de teste
plt.scatter(reg.predict(X_test), reg.predict(X_test) - y_test, color="blue",
↳ s=10, label='Dados de Teste')
# plotando linha para erro residual zero
plt.hlines(y=0, xmin=0, xmax=50, linewidth=2)
# plotando legenda
plt.legend(loc='upper right')
# colocando o título
plt.title("Erro Residual")
```

```
plt.show()
```



No exemplo acima, determinamos a pontuação de precisão usando a Pontuação de Variância Explicada.

Definimos:

$$explained_variance_score = \frac{1 - Var\{y - y'\}}{Var\{y\}}$$

onde: - y' é a saída alvo estimada, - y é a saída alvo correspondente (correta), e - Var é a Variância, o quadrado do desvio padrão.

A melhor pontuação possível é 1.0, valores mais baixos são piores.

1.15 Regressão Linear Polinomial

A Regressão Polinomial é uma forma de regressão linear na qual a relação entre a variável independente x e a variável dependente y é modelada como um polinômio de grau n .

A regressão polinomial ajusta uma relação não linear entre o valor de x e a média condicional correspondente de y , obtida por $E(y|x)$.

1.15.1 Escolhendo um Grau para a Regressão Polinomial

A escolha do grau para a regressão polinomial é um equilíbrio entre viés e variância. - Viés é a tendência de um modelo de prever consistentemente o mesmo valor, independentemente do valor verdadeiro da variável dependente. - Variância é a tendência de um modelo de fazer previsões diferentes para o mesmo ponto de dados, dependendo dos dados de treinamento específicos usados.

Um polinômio de grau mais alto pode reduzir o viés, mas também pode aumentar a variância, levando ao overfitting.

Por outro lado, um polinômio de grau mais baixo pode reduzir a variância, mas também pode aumentar o viés.

Existem vários métodos para escolher um grau para a regressão polinomial, como validação cruzada e o uso de critérios de informação como o Critério de Informação de Akaike (AIC) ou o Critério de Informação Bayesiano (BIC).

1.15.2 Implementação da Regressão Polinomial usando Python

Aqui, importaremos todas as bibliotecas necessárias para tarefas de análise de dados e aprendizado de máquina e, em seguida, carregaremos o conjunto de dados 'Position_Salaries.csv' usando o Pandas.

Em seguida, preparamos os dados para modelagem, lidando com valores ausentes e codificando dados categóricos. Finalmente, dividimos os dados em conjuntos de treinamento e teste e os padronizamos.

```
[21]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn
import warnings
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import KNNImputer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_val_score
warnings.filterwarnings('ignore')
df = pd.read_csv('Position_Salaries.csv')
X = df.iloc[:, 1:2].values
y = df.iloc[:, -1].values

y
```

```
[21]: array([[ 45000,   50000,   60000,   80000,  110000,  150000,  200000,
          300000,  500000, 1000000]])
```

```
[22]: df
```

```
[22]:
```

	Position	Level	Salary
0	Business Analyst	1	45000
1	Junior Consultant	2	50000
2	Senior Consultant	3	60000
3	Manager	4	80000
4	Country Manager	5	110000
5	Region Manager	6	150000
6	Partner	7	200000
7	Senior Partner	8	300000
8	C-level	9	500000
9	CEO	10	1000000

```
[23]: from sklearn.linear_model import LinearRegression
lin_reg=LinearRegression()
lin_reg.fit(X,y)
```

```
[23]: LinearRegression()
```

O código realiza regressão quadrática e cúbica gerando características polinomiais a partir dos dados originais e ajustando modelos de regressão linear a essas características.

Isso permite modelar relações não lineares entre as variáveis independentes e dependentes.

```
[26]: from sklearn.preprocessing import PolynomialFeatures
poly_reg2=PolynomialFeatures(degree=2)
X_poly=poly_reg2.fit_transform(X)
lin_reg_2=LinearRegression()
lin_reg_2.fit(X_poly,y)

poly_reg3=PolynomialFeatures(degree=3)
X_poly3=poly_reg3.fit_transform(X)
lin_reg_3=LinearRegression()
lin_reg_3.fit(X_poly3,y)
```

```
[26]: LinearRegression()
```

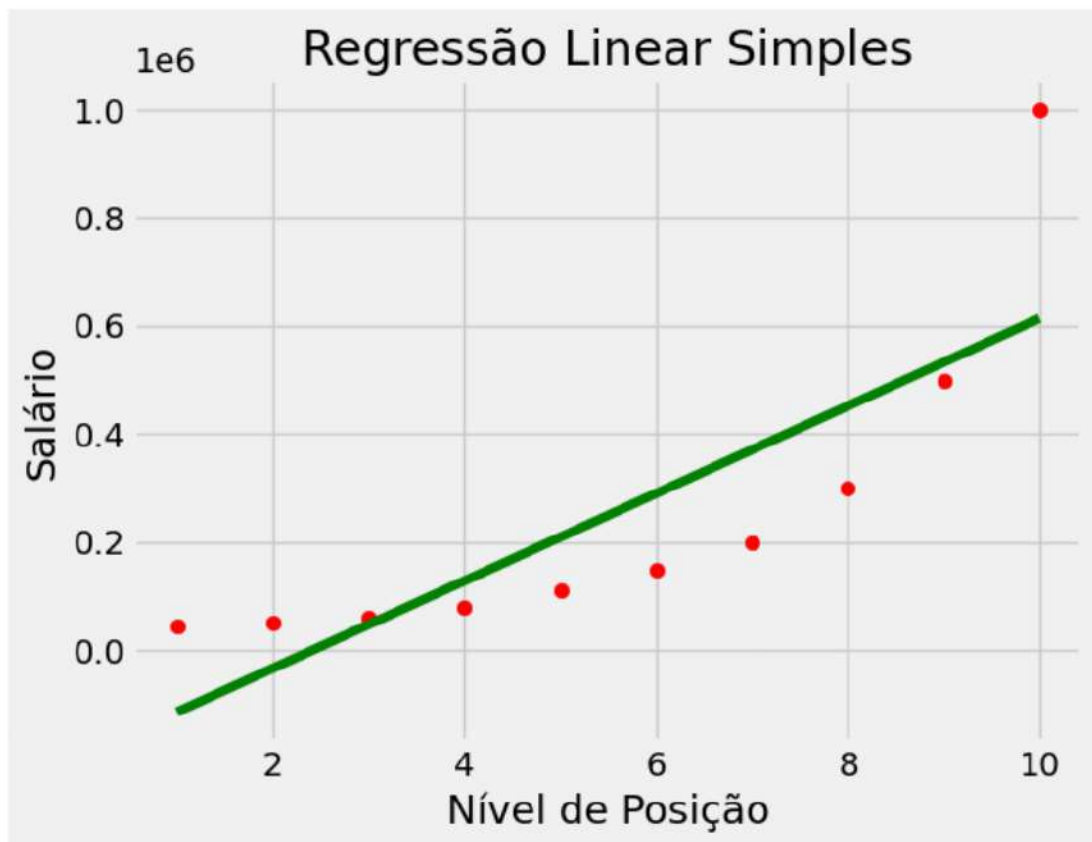
```
[27]: X
```

```
[27]: array([[ 1],
        [ 2],
        [ 3],
        [ 4],
        [ 5],
        [ 6],
        [ 7],
        [ 8],
        [ 9],
```

```
[10]])
```

O código cria um gráfico de dispersão dos pontos de dados, visualizando efetivamente a relação linear entre o nível de posição e o salário.

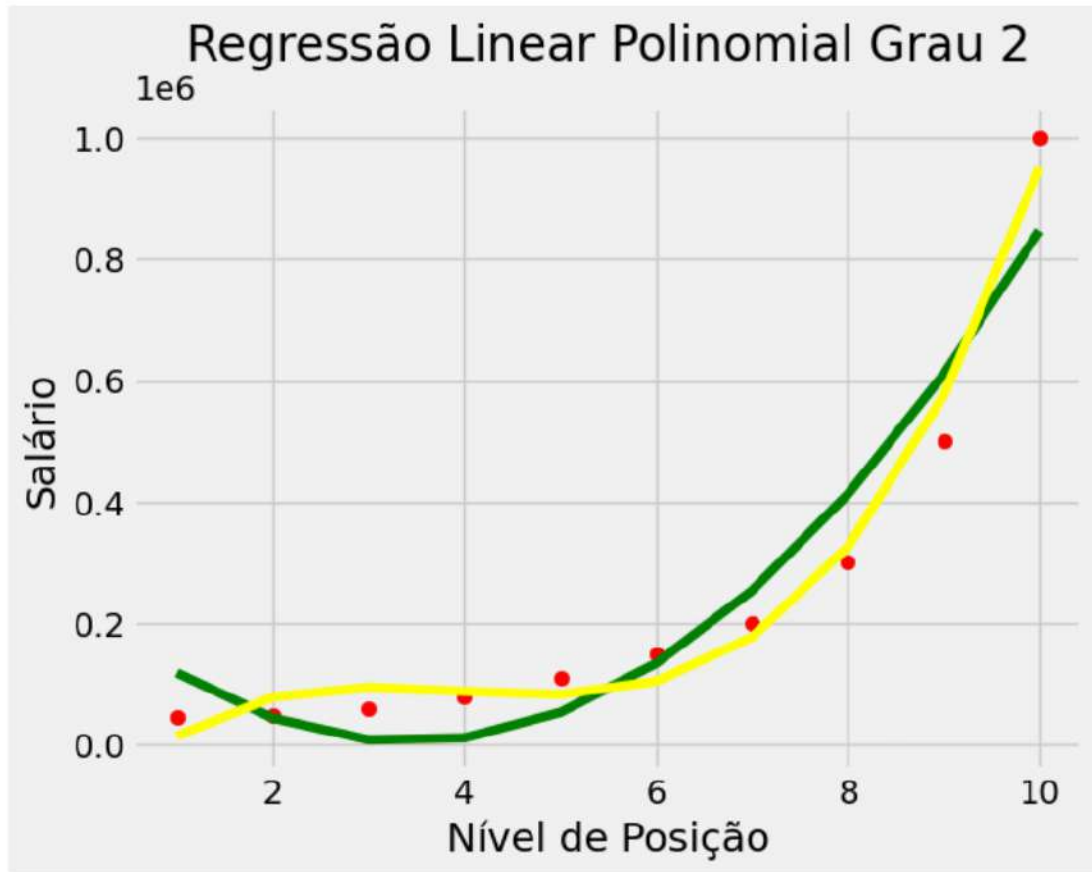
```
[28]: plt.scatter(X,y,color='red')
plt.plot(X,lin_reg.predict(X),color='green')
plt.title('Regressão Linear Simples')
plt.xlabel('Nível de Posição')
plt.ylabel('Salário')
plt.show()
```



O código cria um gráfico de dispersão dos pontos de dados, sobrepondo as linhas de regressão quadrática e cúbica previstas. Ele visualiza efetivamente a relação não linear entre o nível de posição e o salário e compara os ajustes dos modelos de regressão quadrática e cúbica.

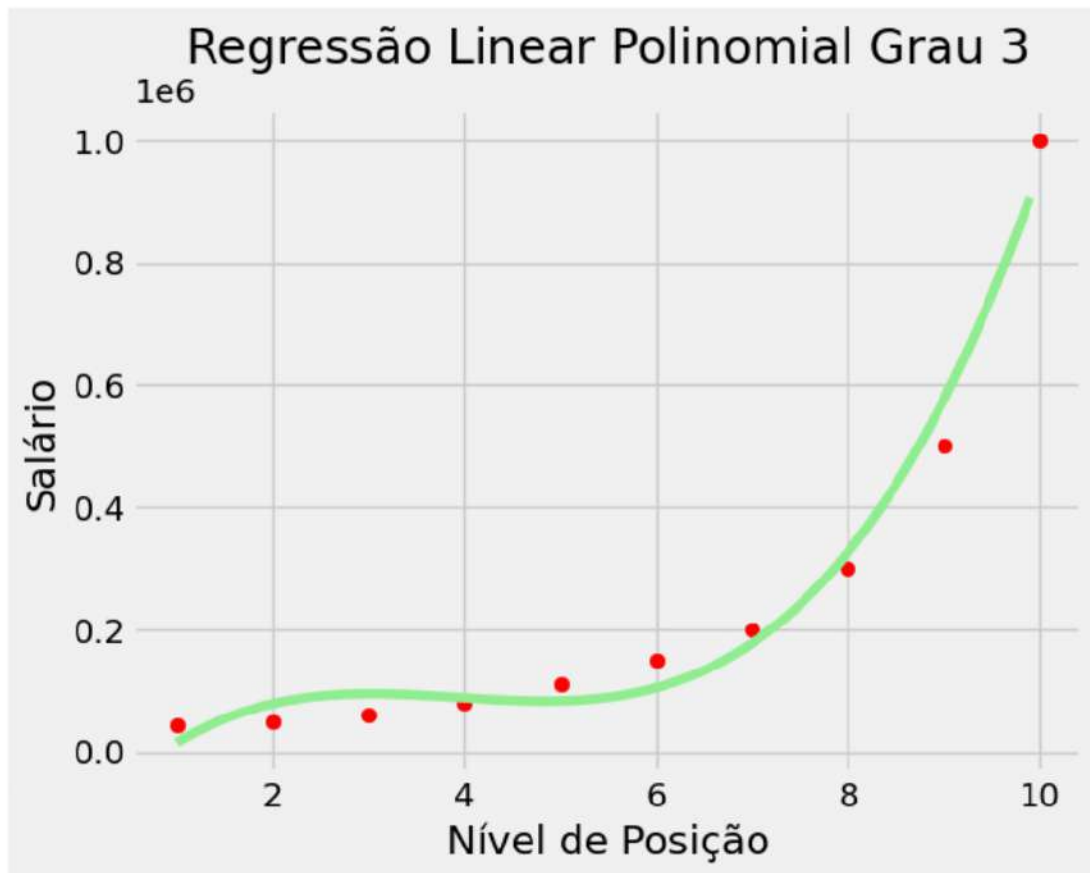
```
[29]: plt.scatter(X,y,color='red')
plt.plot(X,lin_reg_2.predict(poly_reg2.fit_transform(X)),color='green')
plt.plot(X,lin_reg_3.predict(poly_reg3.fit_transform(X)),color='yellow')
plt.title('Regressão Linear Polinomial Grau 2')
plt.xlabel('Nível de Posição')
```

```
plt.ylabel('Salário')
plt.show()
```



O código visualiza efetivamente a relação entre o nível de posição e o salário usando regressão cúbica e gera uma linha de previsão contínua para uma faixa mais ampla de níveis de posição.

```
[30]: X_grid = np.arange(min(X), max(X), 0.1) # Isso nos dará um vetor. Precisamos
      ↪ convertê-lo em uma matriz
      X_grid = X_grid.reshape((len(X_grid), 1))
      plt.scatter(X, y, color='red')
      plt.plot(X_grid, lin_reg_3.predict(poly_reg3.fit_transform(X_grid)),
      ↪ color='lightgreen')
      # plt.plot(X, lin_reg_3.predict(poly_reg3.fit_transform(X)), color='green')
      plt.title('Regressão Linear Polinomial Grau 3')
      plt.xlabel('Nível de Posição')
      plt.ylabel('Salário')
      plt.show()
```



1.15.3 Aplicações da Regressão Linear

- **Linhas de tendência:** Uma linha de tendência representa a variação de dados quantitativos ao longo do tempo (como PIB, preços do petróleo, etc.). Essas tendências geralmente seguem uma relação linear. Portanto, a regressão linear pode ser aplicada para prever valores futuros. No entanto, esse método sofre de falta de validade científica em casos onde outras mudanças potenciais podem afetar os dados.
- **Economia:** A regressão linear é a principal ferramenta empírica na economia. Por exemplo, é usada para prever gastos do consumidor, gastos com investimentos fixos, investimento em estoques, compras de exportações de um país, gastos com importações, demanda por ativos líquidos, demanda por trabalho e oferta de trabalho.
- **Finanças:** O modelo de precificação de ativos de capital usa regressão linear para analisar e quantificar os riscos sistemáticos de um investimento.
- **Biologia:** A regressão linear é usada para modelar relações causais entre parâmetros em sistemas biológicos.

Vantagens da Regressão Linear

1. **Fácil de interpretar:** Os coeficientes de um modelo de regressão linear representam a mudança na variável dependente para uma mudança de uma unidade na variável independente, tornando simples compreender a relação entre as variáveis.
2. **Robusta a outliers:** A regressão linear é relativamente robusta a outliers, ou seja, é menos afetada por valores extremos da variável independente em comparação com outros métodos estatísticos.
3. **Pode lidar com relações lineares e não lineares:** A regressão linear pode ser usada para modelar tanto relações lineares quanto não lineares entre variáveis. Isso porque a variável independente pode ser transformada antes de ser usada no modelo.
4. **Não necessita de escalonamento ou transformação de características:** Diferente de alguns algoritmos de aprendizado de máquina, a regressão linear não requer escalonamento ou transformação de características. Isso pode ser uma vantagem significativa, especialmente ao lidar com grandes conjuntos de dados.

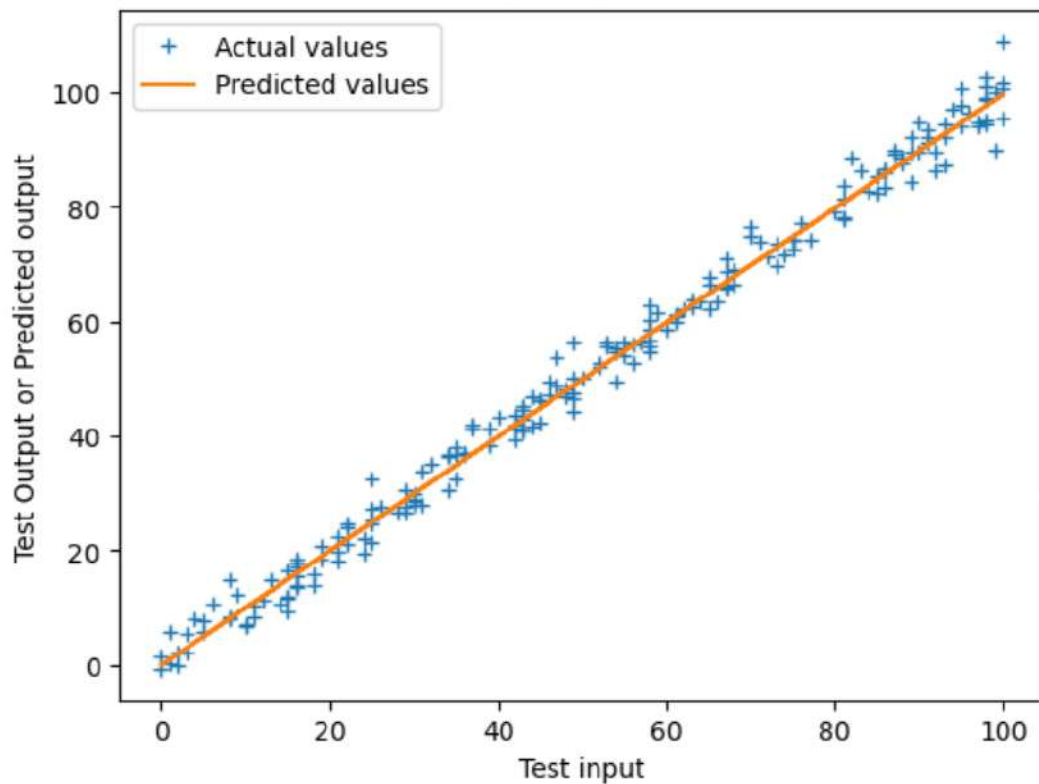
Desvantagens da Regressão Linear

1. **Assume linearidade:** A regressão linear assume que a relação entre a variável independente e a variável dependente é linear. Essa suposição pode não ser válida para todos os conjuntos de dados. Em casos onde a relação é não linear, a regressão linear pode não ser uma boa escolha.
2. **Sensível à multicolinearidade:** A regressão linear é sensível à multicolinearidade. Isso ocorre quando há uma alta correlação entre as variáveis independentes. A multicolinearidade pode dificultar a interpretação dos coeficientes do modelo e levar ao overfitting.
3. **Pode não ser adequada para relações altamente complexas:** A regressão linear pode não ser adequada para modelar relações altamente complexas entre variáveis. Por exemplo, pode não ser capaz de modelar relações que incluem interações entre as variáveis independentes.

Não é adequada para tarefas de classificação: A regressão linear é um algoritmo de regressão e não é adequada para tarefas de classificação, que envolvem a previsão de categorias.

```
[25]: #Prediction on test data
y_pred = test_input*parameters['m'] + parameters['c']

# Plot the regression line with actual data points
plt.plot(test_input, test_output, '+', label='Actual values')
plt.plot(test_input, y_pred, label='Predicted values')
plt.xlabel('Test input')
plt.ylabel('Test Output or Predicted output')
plt.legend()
plt.show()
```



```
[27]: import torch
      from torch.autograd import Variable
```

```
[28]: class LinearRegressionModel(torch.nn.Module):

      def __init__(self):
          super(LinearRegressionModel, self).__init__()
          self.linear = torch.nn.Linear(1, 1) # One in and one out

      def forward(self, x):
          y_pred = self.linear(x)
          return y_pred
```

```
[29]: # our model
      our_model = LinearRegressionModel()
```

```
[31]: criterion = torch.nn.MSELoss(reduction='sum')
      optimizer = torch.optim.SGD(our_model.parameters(), lr = 0.01)
```

```
[33]: import torch
      from torch.autograd import Variable
```

```

x_data = Variable(torch.Tensor([[1.0], [2.0], [3.0]]))
y_data = Variable(torch.Tensor([[2.0], [4.0], [6.0]]))

class LinearRegressionModel(torch.nn.Module):

    def __init__(self):
        super(LinearRegressionModel, self).__init__()
        self.linear = torch.nn.Linear(1, 1) # One in and one out

    def forward(self, x):
        y_pred = self.linear(x)
        return y_pred

# our model
our_model = LinearRegressionModel()

criterion = torch.nn.MSELoss(size_average = False)
optimizer = torch.optim.SGD(our_model.parameters(), lr = 0.01)

for epoch in range(500):

    # Forward pass: Compute predicted y by passing
    # x to the model
    pred_y = our_model(x_data)

    # Compute and print loss
    loss = criterion(pred_y, y_data)

    # Zero gradients, perform a backward pass,
    # and update the weights.
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    print('epoch {}, loss {}'.format(epoch, loss.item()))

new_var = Variable(torch.Tensor([[4.0]]))
pred_y = our_model(new_var)
print("predict (after training)", 4, our_model(new_var).item())

```

```

epoch 0, loss 9.227229118347168
epoch 1, loss 4.3252973556518555
epoch 2, loss 2.13996958732605
epoch 3, loss 1.16404128074646
epoch 4, loss 0.726547360420227
epoch 5, loss 0.5287930965423584
epoch 6, loss 0.43780696392059326

```

epoch 7, loss 0.39439380168914795
epoch 8, loss 0.3722004294395447
epoch 9, loss 0.3594948947429657
epoch 10, loss 0.3510532081127167
epoch 11, loss 0.34455031156539917
epoch 12, loss 0.3389492928981781
epoch 13, loss 0.33378925919532776
epoch 14, loss 0.32886332273483276
epoch 15, loss 0.3240799903869629
epoch 16, loss 0.3193972110748291
epoch 17, loss 0.3147953152656555
epoch 18, loss 0.3102662265300751
epoch 19, loss 0.3058048188686371
epoch 20, loss 0.3014090359210968
epoch 21, loss 0.29707667231559753
epoch 22, loss 0.2928074598312378
epoch 23, loss 0.28859907388687134
epoch 24, loss 0.2844516336917877
epoch 25, loss 0.2803632616996765
epoch 26, loss 0.27633416652679443
epoch 27, loss 0.2723626494407654
epoch 28, loss 0.2684483826160431
epoch 29, loss 0.2645902931690216
epoch 30, loss 0.26078760623931885
epoch 31, loss 0.257039874792099
epoch 32, loss 0.2533455491065979
epoch 33, loss 0.24970465898513794
epoch 34, loss 0.24611611664295197
epoch 35, loss 0.24257905781269073
epoch 36, loss 0.23909294605255127
epoch 37, loss 0.23565661907196045
epoch 38, loss 0.23226985335350037
epoch 39, loss 0.22893193364143372
epoch 40, loss 0.2256416380405426
epoch 41, loss 0.22239899635314941
epoch 42, loss 0.21920251846313477
epoch 43, loss 0.21605247259140015
epoch 44, loss 0.21294745802879333
epoch 45, loss 0.20988701283931732
epoch 46, loss 0.2068704217672348
epoch 47, loss 0.20389749109745026
epoch 48, loss 0.20096710324287415
epoch 49, loss 0.19807900488376617
epoch 50, loss 0.19523216784000397
epoch 51, loss 0.19242626428604126
epoch 52, loss 0.18966075778007507
epoch 53, loss 0.18693524599075317
epoch 54, loss 0.18424880504608154

epoch 55, loss 0.1816006600856781
epoch 56, loss 0.17899082601070404
epoch 57, loss 0.17641834914684296
epoch 58, loss 0.17388293147087097
epoch 59, loss 0.17138409614562988
epoch 60, loss 0.16892091929912567
epoch 61, loss 0.1664932370185852
epoch 62, loss 0.16410058736801147
epoch 63, loss 0.16174213588237762
epoch 64, loss 0.15941756963729858
epoch 65, loss 0.15712657570838928
epoch 66, loss 0.15486830472946167
epoch 67, loss 0.1526428908109665
epoch 68, loss 0.15044912695884705
epoch 69, loss 0.14828675985336304
epoch 70, loss 0.14615565538406372
epoch 71, loss 0.14405518770217896
epoch 72, loss 0.1419849395751953
epoch 73, loss 0.13994431495666504
epoch 74, loss 0.137933149933815
epoch 75, loss 0.13595080375671387
epoch 76, loss 0.13399676978588104
epoch 77, loss 0.13207107782363892
epoch 78, loss 0.1301732212305069
epoch 79, loss 0.1283024102449417
epoch 80, loss 0.12645846605300903
epoch 81, loss 0.12464100122451782
epoch 82, loss 0.12284982949495316
epoch 83, loss 0.12108426541090012
epoch 84, loss 0.11934401094913483
epoch 85, loss 0.11762870103120804
epoch 86, loss 0.11593835055828094
epoch 87, loss 0.11427217721939087
epoch 88, loss 0.1126299798488617
epoch 89, loss 0.11101128160953522
epoch 90, loss 0.1094159334897995
epoch 91, loss 0.10784351825714111
epoch 92, loss 0.10629355162382126
epoch 93, loss 0.10476578772068024
epoch 94, loss 0.10326024889945984
epoch 95, loss 0.10177614539861679
epoch 96, loss 0.1003134548664093
epoch 97, loss 0.09887177497148514
epoch 98, loss 0.097450852394104
epoch 99, loss 0.09605030715465546
epoch 100, loss 0.09466992318630219
epoch 101, loss 0.09330952167510986
epoch 102, loss 0.09196856617927551

epoch 103, loss 0.09064669162034988
epoch 104, loss 0.08934404700994492
epoch 105, loss 0.08805996179580688
epoch 106, loss 0.08679435402154922
epoch 107, loss 0.08554710447788239
epoch 108, loss 0.08431755006313324
epoch 109, loss 0.08310583233833313
epoch 110, loss 0.08191144466400146
epoch 111, loss 0.08073434233665466
epoch 112, loss 0.079573854804039
epoch 113, loss 0.07843039184808731
epoch 114, loss 0.07730318605899811
epoch 115, loss 0.07619214057922363
epoch 116, loss 0.07509731501340866
epoch 117, loss 0.07401801645755768
epoch 118, loss 0.0729542225599289
epoch 119, loss 0.07190564274787903
epoch 120, loss 0.07087232172489166
epoch 121, loss 0.06985369324684143
epoch 122, loss 0.06884987652301788
epoch 123, loss 0.06786029040813446
epoch 124, loss 0.06688512861728668
epoch 125, loss 0.06592389196157455
epoch 126, loss 0.06497647613286972
epoch 127, loss 0.06404253095388412
epoch 128, loss 0.0631222352385521
epoch 129, loss 0.062215112149715424
epoch 130, loss 0.061320848762989044
epoch 131, loss 0.060439612716436386
epoch 132, loss 0.05957101657986641
epoch 133, loss 0.05871492251753807
epoch 134, loss 0.057871051132678986
epoch 135, loss 0.05703924596309662
epoch 136, loss 0.05621951073408127
epoch 137, loss 0.055411726236343384
epoch 138, loss 0.05461529642343521
epoch 139, loss 0.05383029952645302
epoch 140, loss 0.05305676907300949
epoch 141, loss 0.052294373512268066
epoch 142, loss 0.051542773842811584
epoch 143, loss 0.050801970064640045
epoch 144, loss 0.05007186532020569
epoch 145, loss 0.049352265894412994
epoch 146, loss 0.04864298552274704
epoch 147, loss 0.04794386029243469
epoch 148, loss 0.04725493863224983
epoch 149, loss 0.04657584801316261
epoch 150, loss 0.04590630531311035

epoch 151, loss 0.045246630907058716
epoch 152, loss 0.0445963591337204
epoch 153, loss 0.043955378234386444
epoch 154, loss 0.043323732912540436
epoch 155, loss 0.042701058089733124
epoch 156, loss 0.0420873761177063
epoch 157, loss 0.04148260876536369
epoch 158, loss 0.04088633507490158
epoch 159, loss 0.04029877856373787
epoch 160, loss 0.03971962258219719
epoch 161, loss 0.03914878889918327
epoch 162, loss 0.03858627751469612
epoch 163, loss 0.038031600415706635
epoch 164, loss 0.03748510777950287
epoch 165, loss 0.0369463786482811
epoch 166, loss 0.03641531243920326
epoch 167, loss 0.03589203208684921
epoch 168, loss 0.03537611663341522
epoch 169, loss 0.03486776724457741
epoch 170, loss 0.03436656668782234
epoch 171, loss 0.033872656524181366
epoch 172, loss 0.03338589891791344
epoch 173, loss 0.03290611505508423
epoch 174, loss 0.03243328258395195
epoch 175, loss 0.03196711093187332
epoch 176, loss 0.0315077006816864
epoch 177, loss 0.03105483390390873
epoch 178, loss 0.030608531087636948
epoch 179, loss 0.030168607831001282
epoch 180, loss 0.02973511815071106
epoch 181, loss 0.029307790100574493
epoch 182, loss 0.028886601328849792
epoch 183, loss 0.028471382334828377
epoch 184, loss 0.028062211349606514
epoch 185, loss 0.027658920735120773
epoch 186, loss 0.027261465787887573
epoch 187, loss 0.026869595050811768
epoch 188, loss 0.026483401656150818
epoch 189, loss 0.026102904230356216
epoch 190, loss 0.025727706030011177
epoch 191, loss 0.025357987731695175
epoch 192, loss 0.02499350532889366
epoch 193, loss 0.024634316563606262
epoch 194, loss 0.02428029477596283
epoch 195, loss 0.023931315168738365
epoch 196, loss 0.023587409406900406
epoch 197, loss 0.023248497396707535
epoch 198, loss 0.02291429415345192

epoch 199, loss 0.02258499152958393
epoch 200, loss 0.022260379046201706
epoch 201, loss 0.021940529346466064
epoch 202, loss 0.021625138819217682
epoch 203, loss 0.021314401179552078
epoch 204, loss 0.02100803516805172
epoch 205, loss 0.020706096664071083
epoch 206, loss 0.02040862664580345
epoch 207, loss 0.020115215331315994
epoch 208, loss 0.01982617937028408
epoch 209, loss 0.019541259855031967
epoch 210, loss 0.01926039345562458
epoch 211, loss 0.018983643501996994
epoch 212, loss 0.018710821866989136
epoch 213, loss 0.018441902473568916
epoch 214, loss 0.01817687228322029
epoch 215, loss 0.01791560649871826
epoch 216, loss 0.01765814609825611
epoch 217, loss 0.017404329031705856
epoch 218, loss 0.017154179513454437
epoch 219, loss 0.016907749697566032
epoch 220, loss 0.016664771363139153
epoch 221, loss 0.016425250098109245
epoch 222, loss 0.016189105808734894
epoch 223, loss 0.015956535935401917
epoch 224, loss 0.01572715863585472
epoch 225, loss 0.015501143410801888
epoch 226, loss 0.015278365463018417
epoch 227, loss 0.015058813616633415
epoch 228, loss 0.014842323958873749
epoch 229, loss 0.014629075303673744
epoch 230, loss 0.014418786391615868
epoch 231, loss 0.014211567118763924
epoch 232, loss 0.014007402583956718
epoch 233, loss 0.013806087896227837
epoch 234, loss 0.013607688248157501
epoch 235, loss 0.013412045314908028
epoch 236, loss 0.013219315558671951
epoch 237, loss 0.013029361143708229
epoch 238, loss 0.012842074036598206
epoch 239, loss 0.012657509185373783
epoch 240, loss 0.012475650757551193
epoch 241, loss 0.012296339496970177
epoch 242, loss 0.012119561433792114
epoch 243, loss 0.011945411562919617
epoch 244, loss 0.01177375577390194
epoch 245, loss 0.011604541912674904
epoch 246, loss 0.011437775567173958

epoch 247, loss 0.011273354291915894
epoch 248, loss 0.011111372150480747
epoch 249, loss 0.01095164567232132
epoch 250, loss 0.01079428382217884
epoch 251, loss 0.010639164596796036
epoch 252, loss 0.01048626285046339
epoch 253, loss 0.010335568338632584
epoch 254, loss 0.010187019594013691
epoch 255, loss 0.01004064828157425
epoch 256, loss 0.009896312840282917
epoch 257, loss 0.009754068218171597
epoch 258, loss 0.00961390696465969
epoch 259, loss 0.00947576854377985
epoch 260, loss 0.009339573793113232
epoch 261, loss 0.009205352514982224
epoch 262, loss 0.009073054417967796
epoch 263, loss 0.008942659944295883
epoch 264, loss 0.00881409365683794
epoch 265, loss 0.008687460795044899
epoch 266, loss 0.008562600240111351
epoch 267, loss 0.008439539931714535
epoch 268, loss 0.008318224921822548
epoch 269, loss 0.008198713883757591
epoch 270, loss 0.008080878295004368
epoch 271, loss 0.00796474702656269
epoch 272, loss 0.00785029586404562
epoch 273, loss 0.007737454958260059
epoch 274, loss 0.007626289501786232
epoch 275, loss 0.00751668494194746
epoch 276, loss 0.007408588193356991
epoch 277, loss 0.007302194833755493
epoch 278, loss 0.007197211496531963
epoch 279, loss 0.007093772292137146
epoch 280, loss 0.006991825066506863
epoch 281, loss 0.006891330238431692
epoch 282, loss 0.0067923301830887794
epoch 283, loss 0.0066946642473340034
epoch 284, loss 0.0065984525717794895
epoch 285, loss 0.006503644399344921
epoch 286, loss 0.00641015125438571
epoch 287, loss 0.006318051367998123
epoch 288, loss 0.006227264646440744
epoch 289, loss 0.00613776408135891
epoch 290, loss 0.006049534305930138
epoch 291, loss 0.005962631665170193
epoch 292, loss 0.005876913666725159
epoch 293, loss 0.005792454816401005
epoch 294, loss 0.005709190387278795

epoch 295, loss 0.005627139937132597
epoch 296, loss 0.0055463071912527084
epoch 297, loss 0.005466578993946314
epoch 298, loss 0.005387991201132536
epoch 299, loss 0.0053105829283595085
epoch 300, loss 0.005234264302998781
epoch 301, loss 0.005159037187695503
epoch 302, loss 0.0050848727114498615
epoch 303, loss 0.0050117927603423595
epoch 304, loss 0.0049397689290344715
epoch 305, loss 0.0048687816597521305
epoch 306, loss 0.004798823967576027
epoch 307, loss 0.004729843698441982
epoch 308, loss 0.004661863669753075
epoch 309, loss 0.004594852216541767
epoch 310, loss 0.004528822842985392
epoch 311, loss 0.0044637685641646385
epoch 312, loss 0.004399617202579975
epoch 313, loss 0.004336351528763771
epoch 314, loss 0.004274026025086641
epoch 315, loss 0.004212609026581049
epoch 316, loss 0.0041520856320858
epoch 317, loss 0.004092390649020672
epoch 318, loss 0.004033586010336876
epoch 319, loss 0.003975625149905682
epoch 320, loss 0.0039184982888400555
epoch 321, loss 0.003862178884446621
epoch 322, loss 0.003806675784289837
epoch 323, loss 0.0037519452162086964
epoch 324, loss 0.0036980281583964825
epoch 325, loss 0.003644880373030901
epoch 326, loss 0.003592493711039424
epoch 327, loss 0.0035408837720751762
epoch 328, loss 0.0034900051541626453
epoch 329, loss 0.0034398557618260384
epoch 330, loss 0.003390414174646139
epoch 331, loss 0.0033416638616472483
epoch 332, loss 0.003293630201369524
epoch 333, loss 0.00324629875831306
epoch 334, loss 0.0031996783800423145
epoch 335, loss 0.00315369525924325
epoch 336, loss 0.0031083468347787857
epoch 337, loss 0.003063699696213007
epoch 338, loss 0.0030196569859981537
epoch 339, loss 0.0029762820340692997
epoch 340, loss 0.0029334852006286383
epoch 341, loss 0.0028913267888128757
epoch 342, loss 0.0028497744351625443

epoch 343, loss 0.0028088188264518976
epoch 344, loss 0.002768435049802065
epoch 345, loss 0.0027286624535918236
epoch 346, loss 0.0026894700713455677
epoch 347, loss 0.0026508127339184284
epoch 348, loss 0.0026126939337700605
epoch 349, loss 0.002575146034359932
epoch 350, loss 0.0025381483137607574
epoch 351, loss 0.0025016828440129757
epoch 352, loss 0.002465712372213602
epoch 353, loss 0.002430260181427002
epoch 354, loss 0.0023953267373144627
epoch 355, loss 0.002360936952754855
epoch 356, loss 0.0023270035162568092
epoch 357, loss 0.002293543890118599
epoch 358, loss 0.002260581124573946
epoch 359, loss 0.0022280984558165073
epoch 360, loss 0.002196095883846283
epoch 361, loss 0.0021645333617925644
epoch 362, loss 0.0021333922632038593
epoch 363, loss 0.00210275175049901
epoch 364, loss 0.002072518225759268
epoch 365, loss 0.0020427582785487175
epoch 366, loss 0.002013388555496931
epoch 367, loss 0.001984446309506893
epoch 368, loss 0.001955937361344695
epoch 369, loss 0.0019278083927929401
epoch 370, loss 0.0019000985193997622
epoch 371, loss 0.0018728082068264484
epoch 372, loss 0.0018459050916135311
epoch 373, loss 0.001819360302761197
epoch 374, loss 0.0017932343762367964
epoch 375, loss 0.0017674585105851293
epoch 376, loss 0.0017420660005882382
epoch 377, loss 0.001717018662020564
epoch 378, loss 0.0016923510702326894
epoch 379, loss 0.0016680045519024134
epoch 380, loss 0.0016440467443317175
epoch 381, loss 0.0016204211860895157
epoch 382, loss 0.0015971246175467968
epoch 383, loss 0.0015741813695058227
epoch 384, loss 0.0015515575651079416
epoch 385, loss 0.001529252273030579
epoch 386, loss 0.0015072751557454467
epoch 387, loss 0.0014856300549581647
epoch 388, loss 0.0014642680762335658
epoch 389, loss 0.0014432133175432682
epoch 390, loss 0.0014224930200725794

epoch 391, loss 0.0014020291855558753
epoch 392, loss 0.001381884329020977
epoch 393, loss 0.0013620248064398766
epoch 394, loss 0.0013424474745988846
epoch 395, loss 0.001323157805018127
epoch 396, loss 0.0013041547499597073
epoch 397, loss 0.0012854079250246286
epoch 398, loss 0.0012669500429183245
epoch 399, loss 0.0012487077619880438
epoch 400, loss 0.0012307935394346714
epoch 401, loss 0.001213093870319426
epoch 402, loss 0.0011956671951338649
epoch 403, loss 0.001178464386612177
epoch 404, loss 0.0011615367839112878
epoch 405, loss 0.0011448359582573175
epoch 406, loss 0.0011284005595371127
epoch 407, loss 0.0011121693532913923
epoch 408, loss 0.0010961750522255898
epoch 409, loss 0.0010804361663758755
epoch 410, loss 0.0010649114847183228
epoch 411, loss 0.0010496067116037011
epoch 412, loss 0.0010345284827053547
epoch 413, loss 0.0010196431539952755
epoch 414, loss 0.001005005557090044
epoch 415, loss 0.00099055259488523
epoch 416, loss 0.0009763296693563461
epoch 417, loss 0.0009622852667234838
epoch 418, loss 0.000948469853028655
epoch 419, loss 0.0009348271414637566
epoch 420, loss 0.000921390950679779
epoch 421, loss 0.0009081588941626251
epoch 422, loss 0.0008951008203439415
epoch 423, loss 0.0008822375675663352
epoch 424, loss 0.000869559939019382
epoch 425, loss 0.0008570607751607895
epoch 426, loss 0.0008447417058050632
epoch 427, loss 0.0008325974340550601
epoch 428, loss 0.0008206343627534807
epoch 429, loss 0.0008088487666100264
epoch 430, loss 0.0007972042658366263
epoch 431, loss 0.0007857598830014467
epoch 432, loss 0.0007744727772660553
epoch 433, loss 0.000763328280299902
epoch 434, loss 0.0007523634121753275
epoch 435, loss 0.0007415570435114205
epoch 436, loss 0.0007308886852115393
epoch 437, loss 0.0007203864515759051
epoch 438, loss 0.0007100346265360713

epoch 439, loss 0.0006998238386586308
epoch 440, loss 0.0006897817365825176
epoch 441, loss 0.0006798545946367085
epoch 442, loss 0.0006700912490487099
epoch 443, loss 0.0006604627706110477
epoch 444, loss 0.000650969217531383
epoch 445, loss 0.0006416166434064507
epoch 446, loss 0.0006323945708572865
epoch 447, loss 0.000623303116299212
epoch 448, loss 0.0006143322680145502
epoch 449, loss 0.0006055222474969923
epoch 450, loss 0.0005968185141682625
epoch 451, loss 0.0005882398691028357
epoch 452, loss 0.0005797778721898794
epoch 453, loss 0.0005714467260986567
epoch 454, loss 0.0005632420652545989
epoch 455, loss 0.0005551433423534036
epoch 456, loss 0.0005471684853546321
epoch 457, loss 0.0005392986931838095
epoch 458, loss 0.0005315488670021296
epoch 459, loss 0.0005239123129285872
epoch 460, loss 0.00051637995056808
epoch 461, loss 0.0005089611513540149
epoch 462, loss 0.0005016481154598296
epoch 463, loss 0.0004944332176819444
epoch 464, loss 0.0004873373545706272
epoch 465, loss 0.00048033229541033506
epoch 466, loss 0.00047342729521915317
epoch 467, loss 0.0004666249733418226
epoch 468, loss 0.0004599180829245597
epoch 469, loss 0.00045330924331210554
epoch 470, loss 0.000446792459115386
epoch 471, loss 0.0004403667990118265
epoch 472, loss 0.0004340349114499986
epoch 473, loss 0.0004278018022887409
epoch 474, loss 0.0004216603410895914
epoch 475, loss 0.00041559647070243955
epoch 476, loss 0.0004096142074558884
epoch 477, loss 0.0004037324688397348
epoch 478, loss 0.00039793277392163873
epoch 479, loss 0.00039221299812197685
epoch 480, loss 0.000386582687497139
epoch 481, loss 0.00038102484541013837
epoch 482, loss 0.0003755376674234867
epoch 483, loss 0.00037015406996943057
epoch 484, loss 0.000364829262252897
epoch 485, loss 0.00035959493834525347
epoch 486, loss 0.0003544246428646147

epoch 487, loss 0.0003493309486657381
epoch 488, loss 0.0003443053865339607
epoch 489, loss 0.00033935479586943984
epoch 490, loss 0.00033447411260567605
epoch 491, loss 0.0003296783834230155
epoch 492, loss 0.00032493617618456483
epoch 493, loss 0.0003202615771442652
epoch 494, loss 0.0003156611346639693
epoch 495, loss 0.0003111237601842731
epoch 496, loss 0.0003066558565478772
epoch 497, loss 0.00030225058435462415
epoch 498, loss 0.00029789929976686835
epoch 499, loss 0.0002936173987109214
predict (after training) 4 7.980302333831787